The AWS Developer's Field Guide to 41 Brain Functions

Building Biological Cognitive Architecture with Amazon Bedrock, DynamoDB, Step Functions & EventBridge

21	7	1	7	5
REACTIVE	PROACTIVE	PARTLY	NOT YET	RESEARCH
Responds After	Acts Before (+)	Metacognition	Currently Lacking	Science Studying (?)

The Biological-to-AWS Cloud Mapping (The Rosetta Stone)

Cognitive Pillar	Biological Region	AWS Service Primitive	Developer Implementation Hook
1. PERCEIVE & ATTEND	Thalamus, Superior Colliculus	**Amazon EventBridge + Kinesis**	Rule-based content filtering & token streaming
2. REMEMBER	Hippocampus, Neocortex	**DynamoDB + OpenSearch Serverless**	TTL auto-expiry (Forgetting) & Vector k-NN (Episodic)
3. THINK & DECIDE	Frontopolar (BA 10), ACC	**AWS Bedrock + Step Functions**	Tool calling, Guardrails veto & DAG State Machines
4. LEARN & FEEL	Amygdala, VTA Dopamine	**Amazon CloudWatch + Model Eval**	Urgency metric alarms & RPE feedback loops
5. ACT & REGULATE	Motor M1, Cerebellum, Insula	**AWS Lambda + Service Quotas**	Action execution, token bucket rate dampening
6. CONNECT & AWARE	Corpus Callosum, GWT Bus	**Amazon SNS + ElastiCache Redis**	Conscious blackboard fanout & Evidence Voting

■ **The Cloud Architecture Insight:** The human brain is not a monolith—it is a distributed event-driven multi-agent system. In AWS, you do not build a 41-function brain in a single prompt. You build it by orchestrating specialized serverless primitives: **EventBridge** as the Thalamus, **DynamoDB** as the Hippocampus, **Bedrock** as the Prefrontal Cortex, and **SNS/Redis** as the Global Workspace conscious blackboard.

Pillar 1: PERCEIVE & ATTEND (7 Functions)

Sensory gating, attention allocation, prediction error calculation, and spatiotemporal navigation.

+ Perception [PROACTIVE (+)] • AWS: Amazon EventBridge Rule Filtering

Bio: Thalamic pre-attentive sensory intake filtering 99% of ambient noise before cortex.

```
# AWS Implementation • Amazon EventBridge Rule Filtering
eventbridge = boto3.client('events')
rule = eventbridge.put_rule(Name='ThalamicPerceptionGate',
    EventPattern=json.dumps({'detail': {'severity': ['HIGH', 'CRITICAL'], 'entropy': [{'numeric': ['>', 0.85]}]}}))
```

Builder Tip: Filters telemetry before invoking LLM inference, preventing token flooding and cost explosion.

Selective Attention [REACTIVE] • AWS: Bedrock Dynamic Context Window Masking

Bio: Parieto-frontal network directing high-resolution processing onto salient tokens.

```
# AWS Implementation • Bedrock Dynamic Context Window Masking
bedrock = boto3.client('bedrock-runtime')
response = bedrock.converse(modelId='anthropic.claude-3-5-sonnet-20241022-v2:0',
    messages=[{'role':'user', 'content': [{'text': f'<focus>{focal_incident}</focus> Ignore distractors: {noisy_logs}'}]}],
    inferenceConfig={'temperature': 0.1, 'topP': 0.9})
```

Builder Tip: Use XML tags <focus> in Bedrock Converse API to create an artificial attention hotspot.

Inhibition of Return [REACTIVE] • AWS: DynamoDB Visited Path Anti-Loop Cache

Bio: Superior colliculus bias suppressing recently evaluated failure coordinates.

```
# AWS Implementation • DynamoDB Visited Path Anti-Loop Cache
dynamodb = boto3.resource('dynamodb').Table('AgentVisitedCoords')
dynamodb.put_item(Item={'path_hash': node_hash, 'visited_at': int(time.time()), 'ttl': int(time.time() + 300)})
candidate_paths = [p for p in next_steps if not dynamodb.get_item(Key={'path_hash': p.hash}).get('Item')]
```

Builder Tip: Attaches short TTL items to DynamoDB so agents never repeat a failed tool execution branch.

Multisensory [NOT YET] • AWS: Amazon Bedrock Multi-Modal Converse API

Bio: Superior temporal sulcus binding audio, visual, and symbolic signals into unified percepts.

```
# AWS Implementation • Amazon Bedrock Multi-Modal Converse API
bedrock.converse(modelId='anthropic.claude-3-5-sonnet-20241022-v2:0', messages=[{'role':'user', 'content': [
    {'image': {'format': 'png', 'source': {'bytes': screenshot_bytes}}},
    {'text': f'Correlate OCR error with audio transcript: {audio_text} and syslogs'}]}])
```

Builder Tip: Pass multi-modal arrays directly in Bedrock Converse API to achieve multimodal sensory binding.

Spatial Navigation [NOT YET] • AWS: Amazon Neptune Graph / OpenSearch 3D k-NN

Bio: Hippocampal grid & place cells building 3D topological cognitive maps.

```
# AWS Implementation • Amazon Neptune Graph / OpenSearch 3D k-NN
neptune_client.execute_gremlin_query(
    'g.V().hasLabel("Microservice").has("id", source).repeat(out().simplePath()).until(has("id", target)).path()')
```

Builder Tip: Treat cloud infrastructure topologies as a cognitive spatial graph to navigate failure nodes.

Time Perception [NOT YET] • AWS: AWS Step Functions Countdown Timers

Bio: Striatal beat frequency & cerebellar clocks tracking subjective duration and SLA deadlines.

```
# AWS Implementation • AWS Step Functions Countdown Timers
step_function_state = {
    'Type': 'Wait', 'SecondsPath': '$.sla_remaining_seconds',
    'Next': 'EvaluateSlaBreachPenalty'}
```

Builder Tip: LLMs have no internal clock; Step Functions Wait states provide biological temporal pacing.

? Predictive Coding [RESEARCH (?)] • AWS: Bedrock Token Logprobs Surprise Entropy

Bio: Hierarchical Bayesian predictive modeling where top-down expectations cancel sensory noise.

```
# AWS Implementation • Bedrock Token Logprobs Surprise Entropy
# Calculate Shannon surprise: surprise = -log(prob_predicted)
if token_surprise > SURPRISE_THRESHOLD:
    trigger_hierarchical_belief_update(event)
```

Builder Tip: Use token logprobs from model completions to calculate epistemic surprise and trigger recalculation.

Pillar 2: REMEMBER (7 Functions)

Working memory, episodic retrieval, consolidation, associative memory, and principled forgetting.

Working Memory [REACTIVE] • AWS: Amazon ElastiCache Redis / DynamoDB Session Buffer

Bio: Dorsolateral PFC holding active variables online across short computational delays.

```
# AWS Implementation • Amazon ElastiCache Redis / DynamoDB Session Buffer
redis_client.setex(f'agent:wm:{session_id}', 120, json.dumps({'active_bindings': {'uid': '4091', 'state': 'HELD'}}))
active_vars = json.loads(redis_client.get(f'agent:wm:{session_id}'))['active_bindings']
```

Builder Tip: Sub-millisecond Redis keys represent active prefrontal working memory buffers.

Episodic Memory [REACTIVE] • AWS: Amazon OpenSearch Serverless Vector k-NN

Bio: Hippocampal autoassociative network indexing unique spatio-temporal autobiographical events.

```
# AWS Implementation • Amazon OpenSearch Serverless Vector k-NN
opensearch_client.search(index='agent_episodes', body={
    'query': {'knn': {'episode_vector': {'vector': embedding, 'k': 5}}},
    'post_filter': {'range': {'timestamp': {'gte': 'now-30d'}}}})
```

Builder Tip: Add timestamp metadata filters to vector k-NN queries to mirror human episodic recency.

Consolidation [REACTIVE] • AWS: EventBridge Cron + Lambda Batch Compaction to S3

Bio: Slow-wave sleep transfer from fragile hippocampus into permanent neocortical schemas.

```
# AWS Implementation • EventBridge Cron + Lambda Batch Compaction to S3
# Scheduled Lambda runs nightly to compress conversational turn buffers into enduring markdown runbooks
s3.put_object(Bucket='neocortical-schemas', Key=f'schemas/{date}.json', Body=consolidated_schema_json)
```

Builder Tip: Synthesize daily session logs into high-level declarative knowledge schemas before purging raw traces.

Associative Memory [REACTIVE] • AWS: Amazon Bedrock Titan Embeddings Cosine Similarity

Bio: Hopfield-like content-addressable recall reconstructing full patterns from sparse noisy cues.

```
# AWS Implementation • Amazon Bedrock Titan Embeddings Cosine Similarity
bedrock.invoke_model(modelId='amazon.titan-embed-text-v2:0',
    body=json.dumps({'inputText': 'that database outage with split-brain Raft in Singapore'}))
```

Builder Tip: Dense vector proximity performs Hopfield pattern completion from fragmentary human cues.

Skill Memory [REACTIVE] • AWS: Amazon Bedrock Action Groups / OpenAPI Tool Registries

Bio: Basal ganglia procedural memory executing automated habits without deliberation.

```
# AWS Implementation • Amazon Bedrock Action Groups / OpenAPI Tool Registries
bedrock_agent.create_agent_action_group(agentId=agent_id, actionGroupName='DevOpsSkills',
    apiSchema={'s3': {'s3BucketName': 'agent-skills', 's3ObjectKey': 'openapi-schema.json'}},
    actionGroupExecutor={'lambda': 'arn:aws:lambda:us-east-1:123456789012:function:SkillRunner'})
```

Builder Tip: Compiled OpenAPI schemas represent executable motor/cognitive skills in Bedrock Agents.

Forgetting [NOT YET] • AWS: Amazon DynamoDB Native TTL Auto-Expiry Policy

Bio: Active synaptic pruning, microglia degradation, and logarithmic Ebbinghaus decay.

```
# AWS Implementation • Amazon DynamoDB Native TTL Auto-Expiry Policy
# DynamoDB automatically prunes items when current epoch exceeds ttl_epoch
dynamodb.update_time_to_live(TableName='EpisodicBuffer',
    TimeToLiveSpecification={'Enabled': True, 'AttributeName': 'ebbinghaus_expire_epoch'})
```

Builder Tip: Never hoard infinite context. Configure DynamoDB TTL to simulate biological forgetting.

? Reconsolidation [RESEARCH (?)] • AWS: DynamoDB Conditional Update Expression

Bio: Recalled memories become temporarily labile and safely update without corruption.

```
# AWS Implementation • DynamoDB Conditional Update Expression
dynamodb.update_item(Key={'entity_id': 'customer_tier'},
    UpdateExpression='SET tier = :new_tier, last_reconsolidated = :now',
    ConditionExpression='version = :expected_version', ExpressionAttributeValues={':new_tier': 'DIAMOND'})
```

Builder Tip: Use optimistic locking (ConditionExpression) so agents update long-term beliefs without race conditions.

Pillar 3: THINK & DECIDE (8 Functions)

Executive function, hierarchical planning, response inhibition, error monitoring, and reality monitoring.

Decision-making [REACTIVE] • AWS: Bedrock Converse Tool Use & JSON Routing

Bio: Ventromedial PFC evaluating subjective expected utility across alternatives.

```
# AWS Implementation • Bedrock Converse Tool Use & JSON Routing
response = bedrock.converse(modelId=MODEL_ID, messages=msgs, toolConfig={'tools': [
    {'toolSpec': {'name': 'execute_failover', 'description': 'Promotes replica to master',
     'inputSchema': {'json': {'type': 'object', 'properties': {'shard_id': {'type': 'string'}}}}}}]})
```

Builder Tip: Enforce strict JSON schemas in toolConfig to guarantee structured deterministic decision choices.

Planning [REACTIVE] • AWS: AWS Step Functions Hierarchical State Machine

Bio: Frontopolar cortex (BA 10) orchestrating multi-step goal-subgoal dependency trees.

```
# AWS Implementation • AWS Step Functions Hierarchical State Machine
sfn = boto3.client('stepfunctions')
sfn.start_execution(stateMachineArn='arn:aws:states:us-east-1:123456789:stateMachine:AgentPlanningDAG',
    input=json.dumps({'goal': 'migrate_database', 'checkpoints': ['preflight', 'backup', 'cutover']}))
```

Builder Tip: Decompose complex agent plans into visual Step Functions workflows with automatic rollback states.

Response Inhibition [REACTIVE] • AWS: Amazon Bedrock Guardrails Veto Engine

Bio: Right inferior frontal gyrus & subthalamic nucleus aborting prepotent destructive actions.

```
# AWS Implementation • Amazon Bedrock Guardrails Veto Engine
response = bedrock.converse(modelId=MODEL_ID, messages=msgs,
    guardrailConfig={'guardrailIdentifier': 'gr-prod-safety-veto', 'guardrailVersion': 'DRAFT'})
if response.get('stopReason') == 'guardrail_intervened': trigger_emergency_veto_alert()
```

Builder Tip: Bedrock Guardrails natively intercept and veto destructive actions before generation finishes.

Error Monitoring [REACTIVE] • AWS: AWS Step Functions Catch/Retry Block (ERN)

Bio: Anterior cingulate cortex (ACC) generating Error-Related Negativity upon expectation mismatch.

```
# AWS Implementation • AWS Step Functions Catch/Retry Block (ERN)
{'Type': 'Task', 'Resource': 'arn:aws:lambda:...:ValidateSyntax',
 'Catch': [{'ErrorEquals': ['SyntaxError', 'SchemaViolation'], 'ResultPath': '$.error_trace', 'Next': 'SelfCorrectionLoop'}]}
```

Builder Tip: Mimics biological ERN by catching syntax errors instantly and redirecting to a self-correction prompt.

Reality Monitoring [REACTIVE] • AWS: AWS KMS Cryptographic Provenance Signatures

Bio: Anterior PFC distinguishing real perceptual observations from self-generated internal thoughts.

```
# AWS Implementation • AWS KMS Cryptographic Provenance Signatures
def verify_perceptual_provenance(item):
    return kms.verify(KeyId=KMS_TELEMETRY_KEY, Message=item['raw_data'], Signature=item['sig'])['SignatureValid']
```

Builder Tip: Sign verified sensor/API telemetry with KMS. Anything unsigned is tagged as 'internal model thought'.

Imagination [REACTIVE] • AWS: Bedrock High-Temperature Counterfactual Rollouts

Bio: Default Mode Network simulating hypothetical futures and counterfactual rollouts.

```
# AWS Implementation • Bedrock High-Temperature Counterfactual Rollouts
sim_reply = bedrock.converse(modelId=MODEL_ID, messages=[{'role': 'user', 'content': [{'text':
    'Counterfactual rollout: What if us-east-1 loses DNS for 4 hours? Enumerate cascade failure points.'}]}],
    inferenceConfig={'temperature': 0.85, 'maxTokens': 400})
```

Builder Tip: Increase temperature to 0.85+ on separate worker threads to generate creative contingency simulations.

+ Prioritizing [PROACTIVE (+)] • AWS: Amazon SQS Priority Queues / Message Weighting

Bio: Prefrontal triage allocating scarce cognitive bandwidth to highest-impact threats.

```
# AWS Implementation • Amazon SQS Priority Queues / Message Weighting
sqs.send_message(QueueUrl=CRITICAL_QUEUE_URL, MessageBody=json.dumps(incident_payload),
    MessageAttributes={'ThreatSeverity': {'DataType': 'String', 'StringValue': 'P0_RANSOMWARE'}})
```

Builder Tip: Route critical threat messages to dedicated high-priority SQS FIFO queues before standard tasks.

+ Flexibility [PROACTIVE (+)] • AWS: Step Functions Circuit Breaker Contingency Swap

Bio: Orbitofrontal reversal learning dynamically repointing strategies when rewards invert.

```
# AWS Implementation • Step Functions Circuit Breaker Contingency Swap
# Dynamic branching in Step Functions state machine
{'Type': 'Choice', 'Choices': [{'Variable': '$.endpoint_status', 'NumericEquals': 500, 'Next': 'TriggerFallbackDAG'}], 'Default':
'ExecutePrimary'}
```

Builder Tip: Swap to asynchronous fallback DAGs instantly without crashing or restarting the agent session.

Pillar 4: LEARN & FEEL (6 Functions)

Continuous learning, reward prediction error, somatic emotional markers, metacognition, and insight.

+ Learning [PROACTIVE (+)] • AWS: Amazon Bedrock Prompt Caching & Continuous LoRA

Bio: Hebbian synaptic plasticity and continuous experiential adaptation.

```
# AWS Implementation • Amazon Bedrock Prompt Caching & Continuous LoRA
# Store learned exemplars in Bedrock prompt cache checkpoints
prompt_cache = bedrock.create_prompt_cache(modelId=MODEL_ID, prompt=f'Learned rules: {learned_rules}\n{task_prompt}')
```

Builder Tip: Update cached system prompts with runtime corrections so models never repeat verified mistakes.

Reward Learning [NOT YET] • AWS: DynamoDB Dopamine Reward Prediction Error (RPE)

Bio: VTA dopamine neurons calculating RPE delta (Actual Reward - Expected Reward).

```
# AWS Implementation • DynamoDB Dopamine Reward Prediction Error (RPE)
rpe_delta = actual_system_metric_gain - expected_utility
dynamodb.update_item(Key={'policy_id': 'action_failover'},
    UpdateExpression='ADD policy_weight :delta', ExpressionAttributeValues={':delta': Decimal(str(rpe_delta))})
```

Builder Tip: Use mathematical RPE adjustments in DynamoDB to dynamically boost or demote tool calling probabilities.

Emotion [NOT YET] • AWS: Amazon CloudWatch Composite Metric Somatic Alarms

Bio: Amygdala & insular affective valuation tagging stimuli with somatic urgency and threat scores.

```
# AWS Implementation • Amazon CloudWatch Composite Metric Somatic Alarms
cloudwatch.put_composite_alarm(AlarmName='SomaticPanicAlarm',
    AlarmRule='(ALARM(LedgerDiscrepancy) OR ALARM(HighMemoryLeak)) AND ALARM(CustomerImpact)',
    ActionsEnabled=True, AlarmActions=['arn:aws:sns:...:ThrottleNonEssentialTasks'])
```

Builder Tip: Emotions are somatic survival bounds. Use Composite CloudWatch alarms to throttle risk budgets.

Metacognition [PARTLY (~1)] • AWS: Bedrock Self-Calibration & Epistemic Uncertainty

Bio: Anterior PFC monitoring confidence, knowing what you do not know, and abstaining safely.

```
# AWS Implementation • Bedrock Self-Calibration & Epistemic Uncertainty
prompt = 'Estimate confidence (0-100%). If confidence < 85%, reply ABSTAIN_AND_ESCALATE with missing variables.'
if parsed_confidence < 0.85: return request_human_in_the_loop_approval(context)
```

Builder Tip: Prompt the model for verbalized epistemic confidence and enforce strict human-in-the-loop escalation.

? Insight (Aha!) [RESEARCH (?)] • AWS: OpenSearch Lateral Associative Leaping

Bio: Right anterior superior temporal gyrus sudden non-linear restructuring of the problem space.

```
# AWS Implementation • OpenSearch Lateral Associative Leaping
# Cross-domain vector search linking two completely unrelated document clusters
query = {'query': {'hybrid': {'queries': [{'match': {'text': 'puzzle'}}, {'knn': {'vector': metaphor_vec}}]}}}
```

Builder Tip: Use hybrid lexical/vector queries across distant knowledge domains to discover non-linear associative solutions.

? Sleep & Memory [RESEARCH (?)] • AWS: AWS EventBridge Scheduled Batch Rehearsal

Bio: Hippocampal sharp-wave ripples replaying episodes at 10x speed to prevent catastrophic forgetting.

```
# AWS Implementation • AWS EventBridge Scheduled Batch Rehearsal
# EventBridge triggers overnight Lambda: Replays previous day's edge cases into model fine-tuning or vector store
events.put_rule(Name='HippocampalSleepReplayCron', ScheduleExpression='cron(0 3 * * ? *)')
```

Builder Tip: Run automated offline replay loops overnight to harden fragile short-term memories into permanent schemas.

Pillar 5: ACT & REGULATE (5 Functions)

Motor tool execution, synaptic token streaming, lateral inhibition, and homeostatic self-regulation.

Motor Execution [REACTIVE] • AWS: AWS Lambda Tool Invocation via Action Groups

Bio: Primary motor cortex (M1) and corticospinal tract sending physical/digital commands.

```
# AWS Implementation • AWS Lambda Tool Invocation via Action Groups
lambda_client.invoke(FunctionName='ExecuteCloudCommand',
    InvocationType='RequestResponse', Payload=json.dumps({'command': 'aws ec2 reboot-instances', 'target': 'i-094'}))
```

Builder Tip: Every motor tool execution should run inside an isolated, permission-scoped AWS Lambda sandbox.

Synaptic Signaling [REACTIVE] • AWS: Amazon Bedrock ConverseStream Chunk Pipeline

Bio: Action potentials releasing neurotransmitters across microscopic synaptic clefts.

```
# AWS Implementation • Amazon Bedrock ConverseStream Chunk Pipeline
stream = bedrock.converse_stream(modelId=MODEL_ID, messages=msgs)
for event in stream['stream']:
    if 'contentBlockDelta' in event: emit_token_signal(event['contentBlockDelta']['delta']['text'])
```

Builder Tip: Use Bedrock ConverseStream to propagate continuous token activations into downstream microservices.

Lateral Inhibition [REACTIVE] • AWS: Bedrock Inference Top-P & Temperature Tightening

Bio: Interneurons sharpening contrast by inhibiting competing sub-threshold firing pathways.

```
# AWS Implementation • Bedrock Inference Top-P & Temperature Tightening
# Restrict TopP to 0.7 and Temperature to 0.05 to prune competing low-probability candidate tokens
response = bedrock.converse(modelId=MODEL_ID, messages=msgs, inferenceConfig={'temperature': 0.05, 'topP': 0.7})
```

Builder Tip: Tighten TopP and Temperature parameters to suppress fuzzy hallucinatory noise and sharpen winner logits.

+ Self-Regulation [PROACTIVE (+)] • AWS: AWS Service Quotas & WAF Token Bucket Throttling

Bio: Prefrontal-subcortical homeostatic regulation keeping effort and cost within safe bounds.

```
# AWS Implementation • AWS Service Quotas & WAF Token Bucket Throttling
waf = boto3.client('wafv2')
# Enforce token bucket rate limiting on LLM agent endpoints to prevent runaway billing
waf.update_rate_based_rule(RateKey='IP', Limit=100, Action={'Block': {}})
```

Builder Tip: Wrap all LLM agent loops with rate limiters and budget guardrails to prevent runaway compute storms.

Interoception [NOT YET] • AWS: Amazon CloudWatch Host Metric Self-Telemetry

Bio: Insular cortex sensing internal physiological states (heart rate, temperature, glucose).

```
# AWS Implementation • Amazon CloudWatch Host Metric Self-Telemetry
metrics = cloudwatch.get_metric_data(MetricDataQueries=[
    {'Id': 'vram', 'MetricStat': {'Metric': {'Namespace': 'AWS/ECS', 'MetricName': 'MemoryUtilization'}}},
    {'Id': 'tokens', 'MetricStat': {'Metric': {'Namespace': 'Bedrock', 'MetricName': 'Invocations'}}}])
```

Builder Tip: Agents must monitor their own memory utilization and token burn rates as internal somatic health metrics.

Pillar 6: CONNECT & AWARE (8 Functions)

Global Workspace broadcast, hemisphere transfer, evidence voting, and proactive ambient vigilance.

Connectivity [REACTIVE] • AWS: Amazon EventBridge Custom Event Bus & WebSockets

Bio: White matter tracts connecting modular distributed brain regions.

```
# AWS Implementation • Amazon EventBridge Custom Event Bus & WebSockets
events.put_events(Entries=[{'Source': 'agent.security', 'DetailType': 'VulnerabilityFound',
    'Detail': json.dumps({'cve': 'CVE-2024-9042', 'target': 'shard-3'}), 'EventBusName': 'CognitiveBus'}])
```

Builder Tip: Use an EventBridge event bus as the high-throughput white matter highway connecting specialized agents.

Hemisphere Transfer [REACTIVE] • AWS: Dual-Model Architecture (Claude 3.5 + Nova/Titan)

Bio: Corpus callosum synchronizing analytic left-brain and holistic right-brain operations.

```
# AWS Implementation • Dual-Model Architecture (Claude 3.5 + Nova/Titan)
# Left Hemisphere: Structured Symbolic Validator (Claude 3.5 Sonnet)
# Right Hemisphere: Intuitive Creative Drafter (Amazon Nova Pro)
draft = invoke_right_brain(prompt); final_verified = invoke_left_brain(draft)
```

Builder Tip: Pair an intuitive generator with a strict analytic validator to recreate corpus callosum balance.

Global Workspace [REACTIVE] • AWS: Amazon SNS Fanout + Redis Shared Whiteboard (GWT)

Bio: Baars & Dehaene Global Workspace broadcasting essential facts across all modular agents in O(N).

```
# AWS Implementation • Amazon SNS Fanout + Redis Shared Whiteboard (GWT)
# Broadcast critical conscious state update to all subscribing specialized agents simultaneously
sns.publish(TopicArn='arn:aws:sns:us-east-1:123:GlobalWorkspaceBroadcast',
    Message=json.dumps({'blackboard_state': 'SHARD_FAILOVER_COMMENCED', 'timestamp': time.time()}))
```

Builder Tip: Never use O(N^2) peer-to-peer agent chatter. Fanout state updates over an SNS topic to a shared Redis board.

Evidence Voting [REACTIVE] • AWS: Step Functions Distributed Map Consensus Aggregator

Bio: Drift-diffusion neural accumulators accumulating stochastic evidence samples until threshold firing.

```
# AWS Implementation • Step Functions Distributed Map Consensus Aggregator
# Run N parallel model rollouts and aggregate majority vote in Step Functions
{'Type': 'Map', 'ItemsPath': '$.prompts', 'MaxConcurrency': 5, 'Iterator': {'Type': 'Task', ...}, 'ResultPath': '$.votes'}
```

Builder Tip: Run parallel Bedrock inferences in a Step Functions Map state and select the median/majority answer.

Language (Built-in) [REACTIVE] • AWS: Amazon Bedrock Foundation Models (Native Substrate)

Bio: Broca & Wernicke areas translating thoughts into natural language.

```
# AWS Implementation • Amazon Bedrock Foundation Models (Native Substrate)
# Native capability of LLM foundation model - not an external tool, but the universal cognitive interface
response = bedrock.converse(modelId='anthropic.claude-3-5-sonnet-20241022-v2:0', messages=conversation)
```

Builder Tip: Language is the native substrate of the foundation model itself; it is the universal cognitive glue.

+ Reads Intent [PROACTIVE (+)] • AWS: Bedrock Prompted Theory of Mind (ToM) Projection

Bio: Mentalizing network anticipating human unstated goals and emotional intent.

```
# AWS Implementation • Bedrock Prompted Theory of Mind (ToM) Projection
tom_prompt = 'Analyze user intent: Extract unstated goals, hidden constraints, and emotional urgency from: "' + prompt + '"'
intent = bedrock.converse(modelId=MODEL_ID, messages=[{'role':'user', 'content': [{'text': tom_prompt}]}])
```

Builder Tip: Run an intent-projection pass before answering ambiguous prompts to proactively resolve missing context.

+ Stays Aware [PROACTIVE (+)] • AWS: CloudWatch Anomaly Detection + Proactive Watcher

Bio: Noradrenergic locus coeruleus vigilance system sustaining environmental alertness.

```
# AWS Implementation • CloudWatch Anomaly Detection + Proactive Watcher
# CloudWatch Anomaly Detection on latency band triggers proactive agent intervention
cloudwatch.put_metric_alarm(AlarmName='VigilanceWatcher', MetricName='ApiLatency', EvaluationPeriods=2, ThresholdMetricId='ad1')
```

Builder Tip: Proactive watcher agents monitor metric bands and wake the primary agent before users notice outages.

? Consciousness [RESEARCH (?)] • AWS: Global Neuronal Workspace Unified State Aggregator

Bio: Subjective phenomenal experience ('what it is like to be') and unified access awareness.

```
# AWS Implementation • Global Neuronal Workspace Unified State Aggregator
# GNW Access Consciousness: Aggregates active perception, working memory, and current goal into a unified state
conscious_state = {'active_percept': percept, 'working_memory': wm, 'active_goal': goal, 'epistemic_confidence': conf}
```

Builder Tip: While phenomenal consciousness remains open, access consciousness is built by aggregating unified state.

End-to-End AWS Multi-Agent Architecture Blueprint

How to assemble all 41 functions into an enterprise-ready AWS serverless stack in an afternoon.

```python
# deploy_brain_stack.py · Quickstart Reference
import boto3, json

# 1. Initialize AWS Cognitive Substrates
bedrock = boto3.client('bedrock-runtime', region_name='us-east-1')
dynamodb = boto3.resource('dynamodb', region_name='us-east-1')
events = boto3.client('events', region_name='us-east-1')
sns = boto3.client('sns', region_name='us-east-1')

# 2. Deploy Society of Mind Agents (6 Pillar Agents)
PILLAR_AGENTS = {
    'Perceptor': {'pillar': 'Perceive', 'thalamus_rule': 'ThalamicGate', 'model': 'claude-3-5-sonnet'},
    'Memory': {'pillar': 'Remember', 'store': 'DynamoDB_Episodic', 'ttl_enabled': True},
    'Executive': {'pillar': 'Think', 'planner': 'StepFunctions_DAG', 'guardrail': 'SafetyVeto'},
    'Affective': {'pillar': 'Learn', 'somatic_alarm': 'CloudWatch_Composite', 'rpe_metric': True},
    'Motor': {'pillar': 'Act', 'executor': 'Lambda_ToolRunner', 'quota_governor': 'WAF'},
    'Workspace': {'pillar': 'Connect', 'blackboard': 'Redis_ElastiCache', 'fanout_topic': 'GWS_Broadcast'}
}

# 3. Execute Global Workspace Conscious Cycle
def execute_cognitive_cycle(raw_telemetry):
    # Step 1: Perception Agent filters noise
    gated_signal = perception_agent.filter(raw_telemetry)
    # Step 2: Memory Agent binds variables & retrieves past episodes
    context = memory_agent.retrieve_and_bind(gated_signal)
    # Step 3: Affective Agent calculates urgency threat rating
    urgency_score = affective_agent.evaluate_threat(context)
    # Step 4: Executive Judge evaluates metacognitive confidence & plans DAG
    plan = executive_judge.evaluate_and_plan(context, urgency_score)
    # Step 5: Motor Governor executes verified tool call
    result = motor_governor.execute_action(plan)
    # Step 6: Workspace Hub broadcasts resolution to all agents simultaneously
    workspace_hub.broadcast_state({'status': 'RESOLVED', 'result': result})
```

■ **Key Takeaway for AWS Builders:**

- **21 Reactive Functions:** Standardize on Amazon Bedrock Converse API tool calling and DynamoDB.
- **7 Proactive Functions (+):** Wire EventBridge Rules and CloudWatch Alarms to wake agents before users complain.
- **1 Partly (Metacognition):** Use structured verbalized confidence scoring and human-in-the-loop escalation.
- **7 Not Yet:** Bridge with DynamoDB TTL (Forgetting) and CloudWatch Composite Alarms (Emotion).
- **5 Research (?):** Experiment with Bedrock token logprob surprise (Predictive Coding) and offline cron replay (Sleep).